Especificação SDD TaskAnalyzer - Guilherme Fernades Monteiro

Bootcamp III — Especificação Técnica e Governança de Contexto (SDD & AI Harness) — Fase 1

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação

Nome completo: Guilherme Fernades Monteiro

RA (Registro Acadêmico): 22403229

Curso: Ciência de Dados & Machine Learning

Polo/Turma: EAD — Disciplina Virtual (DV)

E-mail institucional: guimonteiro@sempreceub.com

Versão da especificação: 1.0 — Data de elaboração: 13/09/26

1.2 Propósito do Módulo TaskAnalyzer

O TaskAnalyzer é um módulo avançado de métricas e produtividade que analisa um conjunto de tarefas e calcula indicadores de desempenho da execução do trabalho. Seu escopo abrange o cálculo do tempo médio de conclusão das tarefas, a taxa de atraso em relação ao prazo estabelecido e a geração de indicadores agregados por nível de prioridade (baixa, média e alta). O objetivo é transformar dados brutos de tarefas em métricas objetivas que apoiem a tomada de decisão sobre produtividade individual ou de equipe.

1.3 Contrato Executável de Interface

1.3.1 Entradas (Inputs)

Campo | Tipo | Obrigatório | Descrição / Restrições

id_tarefa | int | Sim | Identificador único da tarefa (> 0).

data_criacao | datetime | Sim | Data/hora da criação da tarefa (UTC).

data_inicio | datetime | Não | Data/hora de início da execução.

data_conclusao | datetime | Não | Data/hora de conclusão da tarefa.

prazo | datetime | Sim | Data/hora limite para conclusão.

prioridade | str (enum) | Sim | Valores aceitos: baixa, média, alta.

status | str (enum) | Sim | Valores aceitos: concluído, pendente, cancelado.

1.3.2 Saídas (Outputs)

Campo | Tipo | Descrição

tempo_medio_conclusao | float | Tempo médio de conclusão em minutos/horas, por prioridade e geral.

taxa_atraso_percentual | float | Percentual de tarefas concluídas com atraso, por prioridade e geral.

quantidade_tarefas | int | Total de tarefas analisadas por prioridade (apenas válidas).

indicadores_por_prioridade | dict | Estrutura consolidando as métricas acima agrupadas por prioridade (baixa, média, alta).

1.3.3 Regras de Negócio e Restrições

Considerar apenas tarefas com status "concluído" para o cálculo de tempo médio de conclusão.

O tempo de conclusão é calculado como a diferença entre data_conclusao e data_inicio.

Tarefas com status "cancelado" devem ser ignoradas em todos os indicadores.

Dividir tarefas por prioridade sempre que o indicador for calculado por categoria.

Tratar todas as datas no fuso horário UTC, sem exceções.

Registrar como exceção qualquer entrada com data_conclusao anterior à data_inicio.

2. Especificação de Cenários de Aceite e Test Harness

2.1 Cenários de Aceite (Dado – Quando – Então)

Cenário 1 — Sucesso (Métricas corretas)

Dado: um conjunto de tarefas válidas, com prioridades, prazos e status (concluídas e pendentes).

Quando: o analisador de tarefas for executado.

Então: as métricas retornadas devem estar corretas, incluindo o tempo médio de conclusão por prioridade e geral, a taxa de atraso por prioridade e geral, e a quantidade de tarefas correta por prioridade.

Cenário 2 — Exceção/Erro (Entradas inválidas)

Dado: uma entrada com datas inválidas (ex.: data de conclusão anterior à data de início) ou uma divisão por zero (nenhuma tarefa concluída para calcular a taxa de atraso).

Quando: o analisador de tarefas for executado.

Então: o sistema deve disparar uma exceção específica e tratada, com mensagem clara e objetiva ao usuário.

2.2 Planejamento do Test Harness

Os cenários de aceite descritos acima serão a base para a criação de testes automatizados com pytest na Fase 2, seguindo o fluxo: (1) os Cenários de Aceite descrevem o comportamento esperado do sistema e servem como referência para cada caso de teste; (2) os Casos de Teste decompõem cada cenário em casos específicos, definindo fixtures com dados de entrada preparados; (3) os Testes Automatizados implementam esses casos em pytest, validando entradas, saídas e exceções; (4) a Execução e Validação roda os testes automaticamente a cada alteração no código; e (5) os Relatórios e Evidências geram um relatório de cobertura, garantindo rastreabilidade entre cada cenário e o teste correspondente.

3. Governança de Contexto e Regras para Agentes de IA (CONTEXT_RULES)

3.1 Diretrizes Arquiteturais (Obrigatórias)

Utilizar Python 3.11 ou versão superior como mínimo obrigatório.

Utilizar type hints (anotações de tipo) em todas as funções e métodos.

Seguir o princípio Single Responsibility (SRP) e o padrão de código limpo PEP 8.

Adotar documentação formal no padrão Google style docstrings para módulos, classes, funções e parâmetros.

Manter funções pequenas, coesas e com um único propósito.

Tratar exceções de forma específica, com mensagens claras ao usuário.

Utilizar logs estruturados com o módulo padrão logging.

Escrever código legível, manutenível e com nomes de variáveis e funções descritivos.

3.2 Proibições Explícitas (Regras Restritivas)

Não utilizar bibliotecas externas não autorizadas.

Não alterar, remover ou adaptar os cenários de teste fornecidos nesta especificação.

Não modificar a estrutura de pastas definida nesta especificação.

Não persistir dados em arquivos ou bancos de dados.

Não alterar a assinatura (nome e parâmetros) das funções públicas sem autorização.

Não gerar código sem testes correspondentes para as novas funcionalidades.

Não inserir código duplicado ou sem necessidade.

Não assumir comportamentos não especificados no contrato de negócio.

3.3 Regras de Interação com IA

Fornecer sempre o contexto completo antes de solicitar qualquer geração de código à IA.

Validar se a IA compreendeu corretamente o contrato de negócio e os cenários antes da geração.

Solicitar explicações objetivas sobre qualquer decisão técnica proposta pela IA.

Revisar criticamente todo código gerado, nunca aceitando automaticamente.

Rejeitar e solicitar correção de qualquer resposta que viole as proibições definidas neste documento.

Essas regras compõem o arquivo CONTEXT_RULES.md, que será a fonte de verdade fornecida aos assistentes de IA durante a Fase 2 do projeto.

4. Arquitetura do Repositório e Preparação para Git/GitHub

4.1 Desenho da Árvore do Repositório (Fase 2)

sdd-task-analyzer/ ■■■ README.md # Visão geral do projeto no GitHub ■■■ CONTEXT_RULES.md # Regras persistentes para a IA ■■■ specs/ ■ ■■■ task_analyzer_spec.md # Especificação SDD (baseada nesta Fase 1) ■■■ tests/ ■ ■■■ test_harness.py # Testes automatizados (pytest) ■■■ src/ ■ ■■■ task_analyzer.py # Código

gerado via IA e homologado ■■■ .gitignore # Arquivos e diretórios ignorados ■■■ requirements.txt # Dependências autorizadas

4.2 Plano de Homologação Humana

Na Fase 2, todo código gerado pela IA passará por uma revisão crítica humana antes de ser aceito e versionado, seguindo cinco passos: (1) execução completa dos testes automatizados para validação funcional; (2) revisão de conformidade com o Contrato de Negócio e com as CONTEXT_RULES; (3) análise de qualidade de código, legibilidade, tratamento de erros e aderência aos padrões definidos; (4) testes manuais complementares em casos de borda; e (5) somente após a aprovação em todas as etapas anteriores, o código será commitado no repositório. Esse processo garante que a IA seja utilizada como ferramenta produtiva, mas que a decisão final e a responsabilidade técnica permaneçam sempre humanas.

4.3 Estratégia de Versionamento (Fase 2)

Fluxo de Git simplificado com os branches main, develop e feature/*.

Commits pequenos e descritivos, cada um representando uma unidade lógica de mudança.

Abertura de Pull Requests para toda nova funcionalidade, antes de qualquer merge na branch principal.

Revisões obrigatórias antes de qualquer merge, seguindo o Plano de Homologação Humana.

Uso de tags para marcar releases importantes do projeto TaskAnalyzer.

5. Link da Apresentação em Vídeo

https://youtu.be/GX5s09DNRtI